

the corresponding sender.

4. Text encrypted by a receiver's public key can be decrypted only by the receiver's private key; thus messages encrypted using the public key of the receiver can be decrypted only by the corresponding receiver with the corresponding private key. The https protocol uses this feature.



Figure 1: SSL decryption properties

The https protocol

As discussed in earlier Wireshark articles, it is definitely possible to sniff (eavesdrop) network traffic leading to compromise of sensitive information such as login names, passwords, etc. If clear text protocol is in use. The https protocol is used to guard against possibility of clear text information between client (browser) and server getting compromised.

The https protocol uses the secure socket layer (SSL) or its successor, the transport layer security (TLS) to encrypt traffic between the Web server and the client (browser). SSL, in turn, uses an asymmetric key RSA algorithm for encryption and decryption. When a user sends a browser request to an https website, encrypted communication is established as follows:

1. The browser sends an https request for a secure session towards the server's TCP 443 port (or on a different port for servers running on non-standard ports).
2. The server sends back its digital certificate – which contains its public key.
3. The browser authenticates this certificate with the certifying authority (CA) – the provider of the digital certificate, before continuing further. If the certificate is not authenticated by the CA or self generated, an error like 'This connection is untrusted' is received. This could be a man-in-the-middle (MITM) type of attack, where the attacker makes two independent connections – one with the user and the other with the server, to capture the complete traffic in decrypted format. The user should abort such a connection, or may proceed further only if the origin of the certificate is assured.
4. The client generates a random sequence of characters to be used as a symmetric key.
5. The client encrypts these characters using the server's public key and sends it to the server, thus ensuring that only the corresponding server (or private key) can decrypt it.
6. The server decrypts the symmetric key using its private key.
7. Now, this set of characters is known only to the client and the server. They are used as the symmetric key for

encrypting all further traffic between client and server.

8. The client and the server start encrypted communication using this symmetric key.

You may wonder about the jugglery involved in using a 'hybrid' technique. This is done merely to improve speed since the symmetric key provides much faster encryption and decryption.

Box 1: The importance of the private key and the criticality of Heartbleed

The private key of the server is held confidentially on the server side. So it is obvious that if this key is compromised, it could be used to decrypt sensitive data between the server and the client. It is a primary requirement that this private key is really held privately!

The most popularly used open source software for SSL and TLS is OpenSSL—which was first released in 1998.

Heartbleed could reveal the private key of the server by exploiting a serious vulnerability in OpenSSL. Thus, its arrival was a nightmare for security professionals.

Now, we have sufficient information to understand the capturing and decrypting of https traffic using Wireshark. Start Wireshark and browse any https website—you will definitely notice that the data part of the capture is encrypted. To decrypt data, we must have the private key of the https server.

So, let us proceed with the following steps. This article uses VMware ESXi 5.0 as the target system to decrypt https traffic using Wireshark.

- A. Enable VMware SSH:
 - Use the VMware console keyboard
 - Press the F2 key
 - Provide your login credentials
 - Select troubleshooting options
 - Enable ESXi shellB.
- B. Copy the private key file using the SSH command:

```
ssh root@vmwareesxi:/etc/vmware/ssl/rui.key .
```

- C. Start Wireshark, and let traffic capture begin – you may use capture filter for the VMware ESXi host.
- D. Open the browser, then open the VMware ESXi https Web page and browse a few links.
- E. Capture traffic and stop.
- F. Now, apply the private key to the SSL traffic from this host (as shown in Figure 1):
 - Edit – Preferences
 - Protocols
 - SSL
 - RSA keys list: Edit
 - Insert the required values: IP address, port number, protocol and key file path+name
 - Textbox2 – RSA private key format – RSA key does not require password. The password field is for the file in the PKCS#12 format.